

Static / Program Analysis Engineer: C#.NET static analysis and symbolic-computation tasks at ISP RAS; previously LLVM 22/C++ at MCST. Public work covers Roslyn CFG fixed-point data flow and interprocedural LLVM analysis with conservative applicability checks.

2022: Compilers / VMs · 2023: CIL + native x86-64 · 2024–2025: AST / IR / backends → toolchain · 2026 → : Professional LLVM + static analysis

PROFESSIONAL EXPERIENCE

- 2026 — present

ISP RAS — Static Analysis Engineer

 - Contribute to SharpChecker, ISP RAS's C#.NET static-analysis platform.
 - Current work includes symbolic-computation tasks in C#.NET program analysis.
- July — August 2026

MCST — Compiler Engineering Intern

 - Implemented a LICM pass for LLVM 22 in C++; hoisting conditions check loop structure, memory access, side effects, and speculative safety.
 - Built a Python harness comparing program behavior before/after optimization and checking transform/no-transform cases.

SKILLS

- Data-flow Analysis

Roslyn CFG, worklist/fixed point, edge refinement, predecessor joins, loops/back edges
- Interprocedural / LLVM

effects analysis, affine evolution, APInt, transformation applicability, LLVM IR
- Analysis Correctness

assignment/ref/out invalidation, conservative unsupported-case handling, regression tests, LLVM Verifier
- Languages / Tools

C#, C++23, Roslyn, LLVM 22, .NET

PROJECTS

- DerefAfterNullAnalyzer

A Roslyn ControlFlowGraph analyzer propagates null state on conditional edges to a fixed point, joins predecessor states, and reprocesses successors for loops/back edges.

Method/property/field/index/event dereferences; invalidation after assignment/ref/out; dedicated analyzer tests use Microsoft.CodeAnalysis.CSharp.Testing.
- LLVM Interprocedural Global IV Optimization

An LLVM 22 pass localizes supported induction-like globals after interprocedural effect analysis, affine evolution, and a separate applicability check; APInt models fixed-width wraparound.

29 positive/negative regression cases cover LLVM Verifier checks, structural expectations, idempotence, and observable before/after behavior.

EDUCATION

HSE University — Software Engineering

RECOGNITION

- HSE Faculty of Computer Science — open-source projects competition

Selected as one of the winners with UniversalToolchain / Wist2.
- LangDev'26 — Build the Language, Then Make the Abstractions Disappear

Accepted talk on UniversalToolchain's architecture.
- Earlier distinctions

Two-time overall winner of MEPhI Junior; prize-winner in HSE Vysshaya Proba competitive and industrial programming; First Degree Diploma and Grand Prize at the Baltic Science and Engineering Competition.

Moscow / remote